VibeTesting: An Agentic AI Framework for Automated Test Case Generation

Daniel Egiant Sitanggang
School of Electrical Engineering and Informatics
Bandung Institute of Technology
Bandung, Indonesia
danielegiantsitanggang@gmail.com

Yani Widyani
School of Electrical Engineering and Informatics
Bandung Institute of Technology
Bandung, Indonesia
yani@itb.ac.id

***Abstract*—Software testing is integral to software development, but generating test suites that achieve high code coverage remains difficult. Developers often rely on Large Language Models (LLMs) to generate test suites, though zero-shot approaches often produce incorrect assertions and leave parts of the code uncovered. Existing LLM-based frameworks improve on this by introducing additional generation and refinement steps; however, they can still produce failing and bloated test cases due to continuous addition of the test cases to aim for a higher coverage. CodaMosa is considered one of the state of the art, but it is still susceptible to coverage plateaus and low readability. This paper presents VibeTesting, an agentic AI framework for automated test case generation that combines static program analysis with LLM-based test generation. The framework utilizes control flow graph (CFG) to break down the program structure and identify its execution paths. This information is then used by the LLM to generate and refine the test case further. We evaluate VibeTesting on a benchmark of 69 Python functions derived from TestEval and compare its performance with CodaMosa. The proposed approach achieves 99.48% line coverage, improving on CodaMosa by 3.75 percentage points. Additionally, the framework ensures that the assertions in the final test suite are always in line with the program's behaviour by instructing LLMs to generate only the inputs and capture output from the dynamic executions. Our results demonstrate that iterative agentic architecture, enriched by static program analysis can produce test suites with high coverage and reliability.**

Keywords—Automated Software Testing, Agentic AI, Control-Flow Graph

I. Introduction

Testing is a critical part of the software development life cycle: it validates functionality and ensures reliability. Testing is also known to be highly resource-intensive. Industry data shows that testing consumes 50% to 75% of the development budget, while creating manual test suites accounts for more than 15% of development time [1], [2]. The goals are to test the code bases thoroughly, indicated by high code coverage. The target code coverage is typically set between 80% and 100% [3], [4]. Meeting these targets requires significant investment, hence automated solutions are often desirable.

In this paper, we propose a novel autonomous AI agent system that coordinates a closed-loop cycle of test generation, execution, and refinement. Our key innovation is to integrate the control-flow graph (CFG) with Large Language Models (LLMs), which provides the AI agent with a structural map of the program under test. With this CFG-based context, the agent can analyze uncovered execution paths and target them with high precision. We also separate input generation from assertion validation. This reduces assertion hallucinations and generates highly executable regression test suites.

We make the following contributions:

1) Novel Agentic Architecture: We present a novel architecture of LLM-driven autonomous test generation. This architecture augments LLMs with the program's control-flow graph to help them navigate to each line of code and reach high code coverage. It also separates input discovery from output validation and discards test cases that do not improve the overall coverage of the test suite. This way, the final test suite will always have a 100% execution success rate while keeping the number of test cases small due to the compaction.
2) CFG-Based Guidance: We introduce a technique that translates abstract line coverage gaps into concrete execution paths. This provides structural guidance to LLMs, helping them overcome coverage stagnation.
3) Empirical evaluation: We evaluated our framework on a benchmark comprising 69 complex Python functions. Our framework outperformed the baseline methods evaluated on this benchmark in terms of code coverage, test suite efficiency and execution reliability.

The remainder of this paper is organised as follows. Section II reviews related work on automated software testing (AST) and previous methodologies, the prevalence of LLM-based test generation, the use and the potential of CFG in software testing, and the benchmark of LLM-based AST. Section III describes how we come with VibeTesting architecture, by addressing the gaps and leveraging the potentials of previous studies. Section IV describes our experimental setup and a comparative evaluation of VibeTesting along with the zero-shot approach as a baseline and CodaMosa as the state of the art. Section V discusses empirical insights, threats to validity, and the future directions from this paper. Section VI concludes the paper by summarizing our main contributions.

II. Related Research

A. Automated Software Testing and Previous Methodologies

Automated Software Testing (AST) is defined as testing practice that uses a series of automations [5], [6]. Rather than testing the software manually, AST uses tools to test the software and report the outcome. Recent study has shown that AST allows developers to detect faults earlier and reduce human error in repetitive tasks [6].

Several methodologies that have been researched to perform AST in the past. These methodologies include symbolic execution, model-based testing, combinatorial testing, and search-based software testing (SBST) [7]. SBST is one of the most popular due to its ability in achieving high coverage test suites [8]. EvoSuite is considered as the state of the art for this matter, generating high coverage test suites for Java-based programs. It has its Python adaptation using the same algorithm, Pynguin.

SBST is not perfect and has its limitations. The primary limitation is it could get trapped in a local optima [9]. Since SBST works by mutating previous tests, it could stall when mutating tests have no chance to increase the coverage. This condition is often seen while testing hard-to-reach logical branches [10]. CodaMosa [9] is designed to surmount this problem. It is built on top of SBST (Pynguin) with slight modification. Whenever the SBST engine reaches a coverage plateau, it prompts instruct-LLM to mutate the test cases. These additional test cases help the algorithm to escape the plateau and advance its search.

B. LLM-Based Test Generation and the Agentic Approach

The success of CodaMosa that utilizes LLM might have inspired researchers to realize the potential of LLM for automated software testing. Simultaneously, developers increasingly rely on LLM for test case generation [11]. The users who actively use LLM to generate test cases reportedly see increments in their coverage. In spite of that, LLM-based test generation still has its own limitations: it often hallucinates the assertions and producing a high coverage test suite remains challenging [11], [12].

These limitations inspire the creation of LLM orchestration tools such as CoverUp [13]. CoverUp is an AST tool that is based on an agentic approach. It works by first running the initial existing test suite with a coverage profiler, determining the coverage gaps, then prompting LLM iteratively to fill the coverage gap by refining the test suite. Occasionally, LLM also produces assertion errors in its generated test suite. CoverUp addresses this during the execution of the test suite, where it returns any thrown execution errors to the LLM. LLM then revise their test suite by matching the assertion output with the expected output seen in the error message.

C. Control Flow Graph and Its use in Software Testing

Control flow graph is a representation of all the possible execution paths of a program [14]. It represents every statement in the program as nodes and the logical transition between them as the edges. A program path defined as a finite sequence of connected nodes, starting at the start node and ending at the end node.

CFG is often used in software testing, especially in white-box testing [14], [15] Using Control Flow Graph in software testing works as follows. First the program is being scanned to generate its control flow graph to represent the program. Then, they pick a coverage criterion such as line coverage/statement coverage, branch coverage, or path coverage. The CFG is then used to derive paths to satisfy the chosen criterion, then the test case could be generated from this path. This method helps testers design test cases more efficiently [15].

D. Benchmarking LLMs in Test Case Generation

The use of LLM in automated software testing has been getting more ubiquitous, therefore researchers have been exploring the methodology to assess LLMs' limitations and capabilities. One of the most comprehensive examples is TestEval [12]. It benchmarks varying LLMs to generate unit tests against 210 Python programs. These programs are LeetCode solutions with an average cyclomatic complexity of 13.35.

The benchmark focuses on three tasks: overall coverage generation, targeted line and branch coverage, and targeted path coverage. The overall coverage prompts LLM to generate a suite to cover as much part of the code as possible. The targeted line and branch coverage, asks LLM to generate a test case to cover a specific line or branch. The targeted path coverage requests LLM to generate a test suite that follows an execution path. The paper then evaluates the coverage or accuracy of these tasks. The GPT-4o accuracy declined to 80.97% for targeted line coverage, and 56.67% for targeted path coverage. The study also found the limitation of LLM to produce the correct assertions. The average assertions correctness of tested LLMs is 43.52%, and the GPT-4o sits at 52.99%.

III. The Proposed Framework

Previous studies surface some research gaps and potentials on how we could advance automated software testing (AST) tools. It is evident that the future AST tools are moving towards using LLMs [9], [12], [13]. CodaMosa results show promise in LLM to generate an effective test suite for AST, effectively escaping the coverage plateau. TestEval assesses this hypothesis, evaluating LLMs test case capabilities across multiple tasks. It finds several gaps, including imperfect coverage of each task and low assertion accuracy. In Section III.A., we will describe how these coverage gaps could actually devise a pipeline combination that results in a higher overall coverage. Recent efforts also tried to address the LLMs limitation by employing an agentic approach, such as CoverUp [13]. While we do agree with the high level orchestration, we found some inefficiency that we will address in Section III.B. Additionally, CFG that has been widely used and shown effectiveness in white-box software testing is not used as much in this LLM based approach. Therefore, we propose a novel LLM orchestration architecture to perform AST based on the

Reasoning and Acting (ReAct) paradigm [16]. It uses CFG to generate execution paths that help LLM nafvigate through the program. It will also address several limitations in the previous attempt of the agentic approach.

A. Mathematical Motivation and Expected Coverage

To justify our proposed architecture, we derived a theoretical expected coverage implied by TestEval [12]. Let O (90.23%), L (80.97%), and P (56.67%) represent the overall line coverage, targeted line coverage/ accuracy, and targeted path coverage/ accuracy, respectively. If an LLM-based AST system uses a baseline iterative approach combining overall line coverage to generate initial test suite and refining it by targeting the line gaps, the expected cumulative coverage is modeled as $E_1 = O + (1 - O) \times L$, which yields 98.14%. By incorporating path context that could be derived from the CFG, the expected coverage could be modelled as $E_2 = E_1 + (1 - E_1) \times P$, which yields 99.20%. This result provides the justification and motivation of VibeTesting, which employs both the line-based and path-based refinement strategy.

B. System Architecture and Orchestration

The VibeTesting architecture contains five key modules: The Code Processor, the Test Generator, the Test Runner, the Test Refiner, and the Test Completer, as illustrated by Figure 1. These modules are orchestrated by a state machine. In the concrete implementation, we use the LangGraph framework.

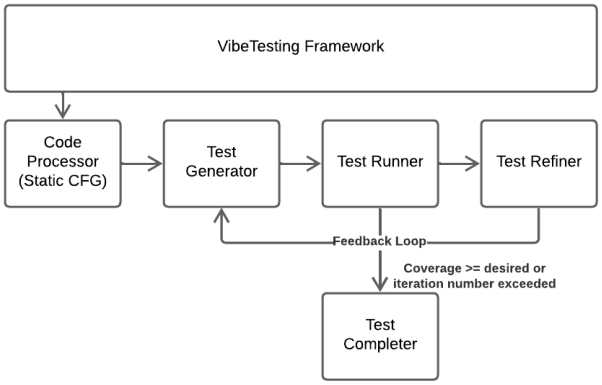


Fig. 1. VibeTesting System Architecture and Orchestration

The Code Processor is the system's static analysis engine. It is responsible for translating the program under test into CFG and execution paths. The use of CFG in this system is inspired by its effectiveness in white-box testing [14] and the mathematical justification we derived in section III.A.

In computing the execution paths, the processor uses Breadth-First Search (BFS) rather than Depth-First Search (DFS). BFS is mathematically preferable because it guarantees discovery of the shortest execution path to any target node or line of code. Minimizing this path length minimizes the number of conditional constraints the LLM must satisfy. Reducing these constraints allows the model to more easily find input that satisfies all conditions. Algorithm 1 shows the algorithm used to find the shortest path to the target node's line number.

Algorithm 1: Shortest Path to Targeted Node

Input: Control-Flow Graph G, Start Node n_{start}, Target Node n_{target}
Output: The shortest sequence of node transitions to reach the target

```
visited ← {n_start}
path_queue ← queue([[n_start]])
while path_queue is not empty do
    current_path ← path_queue.dequeue()
    current_node ← current_path.last()
    if current_node == n_target then
        return current_path
    end if
    for each neighbour ∈ G.adj_list[current_node] do
        if neighbour ∉ visited then
            visited ← visited ∪ {neighbour}
            new_path ← current_path + [neighbour]
            path_queue.enqueue(new_path)
        end if
    end for
end while
return null
```

A major architectural weakness of the existing agentic AST tools (e.g., CoverUp) is their reliance on the LLM to generate the test inputs and expected assertion outputs simultaneously (e.g., `assert SUT(inputs) == expected`). Since LLMs must autoregressively guess the internal execution state and output values, this architecture often experiences assertion hallucinations, causing test case execution to fail. The LLM will either revise this in the subsequent turn, introducing token and time overhead, or worse, it will be completely discarded from the test suite, even though it traverses new lines of code.

VibeTesting solves this bottleneck through execution decoupling. During the generation and refinement loops, our architecture restricts the LLM's role to generating only the program input. The LLM is prompted to output only raw, assertion-free function calls with concrete execution parameters (e.g., `categorize_by_age(5)`).

Another weakness we found in the current agentic AST tools is the way they refine the test suite. First, it asks LLM to revise the whole test suite whenever there is a coverage gap. LLM often addresses these gaps by adding more test cases. Often, the test cases produced by the LLM are ineffective to address those gaps, while still increasing the size of the test suite. These additional test cases could also be redundant with previous test cases meaning it explores no additional statements that have been explored by another test case . This could cause the test suite to be unnecessarily large.

VibeTesting addresses this weakness by introducing a compaction mechanism in the Test Runner. In every execution of the test suite, the system will only retain the test case only if it executes a statement that has not been covered by any other test cases in the test suite. Therefore, any test cases that fail to contribute to the overall coverage will automatically be pruned. The pseudocode of this compaction algorithm can be seen in Algorithm 2.

Once the line-coverage target is reached or the maximum number of turns is exceeded, the system advances to the Test Completer. The Test Completer transforms the test cases that

are still in the form of function calls into a complete Pytest test suite. Test Completer generates test assertions by fetching the return of each function call programmatically, using Python's exec and eval function. These results are then injected into standard Pytest assertions through a predefined template. This design completely eliminates incorrect assertions that are often seen in LLM-based test generation and guarantees a 100% execution success rate.

Algorithm 2: Test Suite Execution and Compaction

Input: List of LLM-generated test cases T_{gen}, SUT source code S
Output: Compacted, non-redundant list of verified tests T_{ver}

```
covered_lines ← ∅
T_ver ← []
for each test ∈ T_gen do
    executed_lines ← run_and_analyze(S, test)
    prev_count ← |covered_lines|
    covered_lines ← covered_lines ∪ executed_lines
    if |covered_lines| == prev_count then
        continue  // Redundant; no coverage gain
    else
        T_ver.append(test)
    end if
end for
return T_ver
```

C. Finding the Best Workflow

To determine the final state machine of the architecture, we explored and evaluated two workflows. One workflow is Exhaustive Generation + Missing-Line Refinement (Workflow 1), and the other is CFG-Based Generation + CFG-Based Refinement (Workflow 2).

Workflow 1 relies on the raw capability of the LLM. We prompt the Test Generator to produce an exhaustive set of initial tests based solely on the program under test. If coverage is incomplete, the Test Refiner sends the list of uncovered line numbers back to the LLM.

The second workflow provides the LLM with a static analysis of the program derived from the CFG. The Test Generator explores execution paths that could reach all parts of the code. If there are any missing lines from the test suite, the Test Refiner uses the CFG to compute the exact execution path needed to reach an uncovered line. Then, it sends this path to the LLM as a constraint.

We benchmarked the two workflows using 69 Python problems that have an average of 35.32 lines of code and 13.42 cyclomatic complexity. Workflow 1's initial test suite achieved an average coverage of 97.50%. After three iterations of the missing-line refiner, coverage increased to 98.63%, 98.82%, and 99.00%, respectively. Workflow 2 achieved an initial average coverage of 91.02%. The three subsequent iterations of the CFG-Based Refinement increased the coverage to 95.04%, 95.42%, and 95.60%, respectively.

Since neither workflow is optimal on its own, we designed a multi-stage hybrid workflow that leverages the strengths of both. It runs test generation in three sequential phases:

1) Initial Generation: First, the Exhaustive Generator is run to obtain high initial coverage.
2) Primary Refinement: The state machine advances to the Missing-Line Refiner, providing LLM with the line number of missing the statements.
3) Secondary Refinement: The state machine continues with the CFG-Based Refiner to provide execution paths to reach the remaining missing statements and bring them to the LLM.

This ordering allows the system to produce high initial coverage and maximize the increase in coverage through both refinement mechanisms. We used this final workflow as the state-machine configuration of our architecture.

IV. Experiment and Analysis

We have conducted a comprehensive empirical evaluation to validate the performance of VibeTesting in generating test suites. This experiment was designed to measure its effectiveness to produce relevant test cases to traverse through the program under test. We also compare VibeTesting against established baselines and the current state-of-the-art on a challenging dataset.

A. Experimental Setup

We evaluated our framework using 69 Python functions derived from the TestEval benchmark. First, we transformed the TestEval Python dataset programmatically into basic Python functions. This enabled the VibeTesting implementation to process each code with PyCFG. Then, we only considered functions with a cyclomatic complexity greater than 10. We computed the cyclomatic complexity of each program using the Radon Python library.

We compared the framework against two baselines representing different paradigms in automated test generation. The first is a zero-shot LLM approach using GPT-4o. This approach represents the capabilities of a state-of-the-art (SoTA) model when applied naively without additional orchestration. The second baseline is CodaMosa, which we selected to represent the current SoTA tool. We ran CodaMosa using its standard configuration: a 120-second time budget and the GPT-3.5-turbo-instruct model.

Although CoverUp is a prominent SoTA tool for LLM-based test generation, we did not include it in the empirical comparison. The reason is the constraint of CoverUp's code processing that is designed to process methods defined inside classes. Our dataset, which consists of standalone Python functions, is unable to be processed by CoverUp. Therefore, we restrict the empirical comparison to CodaMosa and the zero-shot baseline and analyze CoverUp's methodology in Section II.

We used four key metrics in comparing each architecture: Line Coverage, Execution Success, Effectiveness, and Execution Time. Line coverage, that is defined as the percentage of executable source lines covered by the generated test suite, is selected as the primary metric. Execution Success is a percentage of test cases that compile and run without syntax or runtime errors. Effectiveness measures the effectiveness of a test suite, computed as the ratio of the total number of unique lines covered to the total number of generated test cases. A higher effectiveness score indicates a concise test suite implying low maintenance cost. Execution time represents the total duration of running the test suite

which is a proxy of measuring the computational cost of maintaining the test suite.

VibeTesting uses OpenAI's GPT-4o for all agentic components. We left the decoding parameters at their defaults: a temperature of 1.0, a top-p of 1.0, and a maximum output of 4,096 tokens. We did not restrict input tokens so that the models can receive the full context window. In this benchmark, VibeTesting uses Exhaustive Generator + 3x Missing-Line Refiner + 2x CFG-Based Refiner. The complete source code, the benchmark dataset, and the full LLM prompt templates are publicly available at https://github.com/egijago/vibe-testing.

B. Final Workflow Performance

Our benchmark reveals the incremental test suite coverage of VibeTesting, as shown in Figure 2. The Exhaustive Generator alone achieved a coverage of 97.5%. Each of the 3x Missing-Line Refiners raised the overall test suite coverage to 98.63% (+1.13 points), 98.82% (+0.19 points), and 99.00% (+0.18 points), respectively. Each of the 2x CFG-Based Refiners improved this coverage further to 99.33% (+0.33 points) and 99.48% (+0.15 points). The Missing-Line Refiner covers most of the missing-line gaps while showing a diminishing coverage increment after subsequent turns. The CFG-based Refiner, potentially escapes this plateau/ diminishing increment, as shown in the recovering increment by the first CFG refiner. This result highlights the contribution of each component and shows that the two refiners are conceivably complementary.

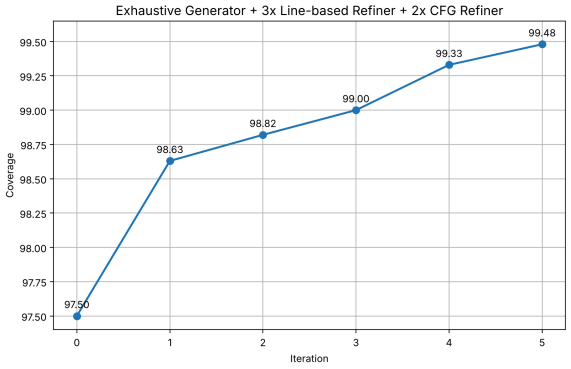


Fig. 2. Incremental Coverage Gain of VibeTesting

C. Comparative Performance Analysis

In our evaluation, the proposed architecture outperformed the zero-shot baseline and the SoTA tool, CodaMosa. The results of each are presented in Table I. Our benchmark shows that VibeTesting achieved 99.48% line coverage, covering 1,932 lines and missing only 10. CodaMosa, on the other hand, reached 95.73% (1,859 lines covered and 83 missed), putting our framework 3.75 percentage points ahead. Both systems are substantially better than the zero-shot GPT-4o baseline, which achieved only 81.10% line coverage and missed 367 lines.

All 236 test cases generated by our framework executed successfully with no syntax or runtime failures, resulting in a 100% execution success rate. This resulted from our architectural decision to decouple logical input generation from assertion creation. The zero-shot baseline was less reliable, with 205 of its 666 test cases failing for an execution success rate of 69.21%. CodaMosa produced 15 failing test cases for an execution success rate of 95.10%.

VibeTesting is also producing more effective test cases with an Effectiveness score of 8.186. It achieved the highest overall coverage with the fewest test cases: 236 successful test cases, against 291 for CodaMosa and 461 for the zero-shot baseline. This comes from the Test Runner's compaction algorithm, which evaluates coverage improvements sequentially and keeps a generated test case only if it expands the union of covered lines. The resulting test suite also executes quickly, in 0.55 s, compared with 3.72 s for the zero-shot baseline and 17.34 s for CodaMosa.

TABLE I. BENCHMARK RESULTS

Metrics	***Zero-Shot LLM***	***CodaMosa***	***VibeTesting***
Line Coverage	81.10%	95.73%	**99.48%**
Execution Success	69.21%	95.10%	**100%**
Effectiveness	2.364	6.075	**8.186**
Total Function Calls	666	806	236
Successful Test Cases	461	291	236
Failed Test Cases	205	15	0
Covered Lines	1575	1859	**1932**
Missing Lines	367	83	**10**
Execution Time	3.72s	17.34s	0.55s

V. DISCUSSION

Our benchmark confirms the efficacy of VibeTesting. In this section, we interpret the results and discuss their implications for AI in software engineering. We also outline the limitations of the study and a path for future research.

A. Why the CFG-Guided Agentic Approach Excels

The agent operates within an iterative perceive-reason-act-learn cycle, enabling it to learn from its own actions. When the initial test suite does not achieve full coverage, the agent identifies the missing line gap, and further uses the CFG to generate execution paths to reach each line. This method allows the system to mitigate the "coverage plateau" problem that often seen in the less-guided methods.. Our finding also could be seen as a proof of concept of a hybrid system, integrating traditional static analysis with LLMs.

B. Threats to Validity and Future Directions

Several threats to validity affect how our empirical findings should be interpreted and how far they generalize. The primary internal threat is the nondeterminism of LLMs. Due to computational budget constraints, we run the benchmark only in a single run. However, LLMs tend to vary across multiple independent runs. Therefore, in order to have a more comprehensive evaluation, one should assess the agent's stochasticity. Additionally, the 100% execution success rate

reported applies specifically to the final, compacted test suite after filtering by the Test Runner. Although it confirms high executability it does not measure the success of executing raw, unrefined LLM generations.

The issue for construct validity is our primary evaluation metric, line coverage. Although line coverage is an established standard in test generation benchmarks, high statement coverage does not guarantee exhaustive fault-detection capability. Our ability to evaluate comprehensively is limited because we do not report branch coverage or mutation analysis.

Our architecture also does not solve the classical test oracle problem. Pytest assertions are compiled by capturing the current runtime behavior of the program under test programmatically. Thus, the generated test suites primarily serve as regression oracles. They can reliably detect behavioral regressions caused by later code modifications. However, they cannot verify whether the current implementation complies with functional requirement specifications.

External threats concern the scope of the benchmark and infrastructure coupling. First, we limited the benchmark to 69 standalone Python functions to evaluate the path-refinement engine under controlled conditions. Second, we acknowledge that there is a discrepancy between the model's inference quality in our benchmark. CodaMosa requires an Instruct-tuned model (GPT-3.5-Turbo-Instruct) and cannot operate on Chat-tuned models, such as GPT-4o. To keep the comparison fair, we selected the strongest available instruct model, that is GPT-3.5-Turbo-Instruct. Therefore, the performance difference may stem from the underlying LLM capabilities rather than the architecture alone.

We highlight two directions for future research. First, in order to scale the framework to be practical and widely adopted, we need to extend the scope of the system to be able to handle multiple modules and classes. Second, the missing lines left by CodaMosa and VibeTesting in our benchmarks are rather complementary. Merging the CodaMosa and VibeTesting test suites will achieve 99.74% line coverage which is exceptionally high. This result surfaces the potential of a hybrid system combining SBST with agentic-based AST tools.

VI. Conclusion

This paper explored a new framework to generate high coverage test suites. We addressed the gaps from the existing agentic tool by decoupling the input and output generation and introducing a test compaction mechanism. We also identified the potentials of using CFG from the previous studies, then leveraged it in our framework. We then designed a rigorous benchmark consisting of only complex Python functions derived from the TestEval dataset. An empirical evaluation is then produced by testing VibeTesting, CodaMosa, and zero-shot baseline against the benchmark. Our empirical evaluation shows the efficacy of our framework in producing high coverage test suites by achieving a coverage score of 99.48, surpassing CodaMosa 3.75 points. Our design decisions also allow VibeTesting to ensure 100% while maintaining the test suite compact. Our works also demonstrate a synergy between agentic architecture with static program analysis.

References

[1] T. G., "The economic impacts of inadequate infrastructure for software testing," National Institute of Standards and Technology, Gaithersburg, MD, USA, 2002.

[2] E. Daka and G. Fraser, "A Survey on Unit Testing Practices and Problems," in *2014 IEEE 25th International Symposium on Software Reliability Engineering*, Naples, Italy: IEEE, Nov. 2014, pp. 201–211. doi: 10.1109/ISSRE.2014.11.

[3] C. Steve, "Minimum Acceptable Code Coverage," Bullseye Testing Technology. Accessed: Oct. 07, 2026. [Online]. Available: https://www.bullseye.com/minimum.html

[4] Carlos Arguelles, M. Ivanković, and A. Bender, "Code Coverage Best Practices," Google Testing Blog. Accessed: Oct. 07, 2026. [Online]. Available: https://testing.googleblog.com/2020/08/code-coverage-best-practices.html

[5] E. Dustin, J. Rashka, and J. Paul, *Automated software testing: introduction, management, and performance*. Reading, Mass: Addison-Wesley, 1999.

[6] V. Garousi and M. V. Mäntylä, "When and what to automate in software testing? A multi-vocal literature review," *Inf. Softw. Technol.*, vol. 76, pp. 92–117, Aug. 2016, doi: 10.1016/j.infsof.2016.04.015.

[7] S. Anand *et al.*, "An orchestrated survey of methodologies for automated software test case generation," *J. Syst. Softw.*, vol. 86, no. 8, pp. 1978–2001, Aug. 2013, doi: 10.1016/j.jss.2013.02.061.

[8] Y. Tang, Z. Liu, Z. Zhou, and X. Luo, "ChatGPT vs SBST: A Comparative Assessment of Unit Test Suite Generation," *IEEE Trans. Softw. Eng.*, vol. 50, no. 6, pp. 1340–1359, Jun. 2024, doi: 10.1109/TSE.2024.3382365.

[9] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, Melbourne, Australia: IEEE, May 2023, pp. 919–931. doi: 10.1109/ICSE48619.2023.00085.

[10] T.-M.-A. Bui, V.-T. Do, M. V. Le, and Q.-T. Bui, "Dynamic Difficulty Coefficient in Search-Based Software Testing: Targeting to Hard Branch Coverage," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, Melbourne VIC Australia: ACM, Jul. 2024, pp. 711–714. doi: 10.1145/3638530.3654411.

[11] M. D. Santana, C. Magalhaes, and R. de S. Santos, "Software Testing with Large Language Models: An Interview Study with Practitioners," Oct. 20, 2025, *arXiv*: arXiv:2510.17164. doi: 10.48550/arXiv.2510.17164.

[12] W. Wang *et al.*, "TESTEVAL: Benchmarking Large Language Models for Test Case Generation," in *Findings of the Association for Computational Linguistics: NAACL 2025*, Albuquerque, New Mexico: Association for Computational Linguistics, 2025, pp. 3547–3562. doi: 10.18653/v1/2025.findings-naacl.197.

[13] J. Altmayer Pizzorno and E. D. Berger, "CoverUp: Effective High Coverage Test Generation for Python," *Proc. ACM Softw. Eng.*, vol. 2, no. FSE, pp. 2897–2919, Jun. 2025, doi: 10.1145/3729398.

[14] P. Ammann and J. Offutt, *Introduction to software testing*. New York: Cambridge University Press, 2008.

[15] W.-C. Pai, C.-M. Chung, C.-T. Hsieh, C.-C. Wang, and Y.-H. Wang, "Software Testing Methodology with the Control Flow Analysis," *J. Inf. Sci. Eng.*, vol. 21, pp. 1213–1226, 2005.

[16] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," in *International Conference on Learning Representations (ICLR)*, Kigali, Rwanda: arXiv, May 2023. doi: 10.48550/arXiv.2210.03629.